UNIT 6

Spark ML Programming using PySpark

Lecture (with Titanic Case Study)

Course Learning Outcomes

After completing this unit students will:

- Understand distributed ML concepts
- Implement ML algorithms using PySpark
- Build end-to-end ML pipelines
- Evaluate and compare models
- Perform complete industry-style case study

Agenda

1. ML Foundations
2. Spark ML Architecture
3. Data Preprocessing
4. Titanic Case Study (Classification)
5. Regression Example
6. Clustering Example
7. Evaluation Metrics
8. Lab Practical

What is Machine Learning?

Machine Learning enables systems to:

- Learn patterns from data
- Make predictions
- Improve automatically

General Function:

Model = f(Features)

Types of Machine Learning

Supervised Learning

- Regression
- Classification

Unsupervised Learning

- Clustering

Why Spark for ML?

Big Data Challenges:

- Large volume
- Distributed storage
- Need parallel computation

Spark provides:

- In-memory processing
- Distributed ML
- Fault tolerance

Spark ML Architecture

Core Components:

- DataFrame: rows and columns, similar to a spreadsheet or SQL table.
- Transformer: A component that transforms data from one representation to another (feature engineering/ StandardScaler).
- Estimator: object that can be fit to data to learn parameters (LinearRegression).
- Model: trained version of an estimator after calling fit()
- Pipeline: A workflow that chains multiple steps together: data preprocessing, feature transformation, and model training.

Terminology

DataFrame = raw materials (your dataset).

Transformer = machines that shape the materials (feature preprocessing).

Estimator = the machine that learns how to assemble them (training).

Model = the finished product (trained predictor).

Pipeline = the entire factory workflow, ensuring everything runs smoothly in sequence.

Transformer

Input → DataFrame
Output → Transformed DataFrame

Examples:

- StringIndexer: Converts categorical string labels into numeric indices.
- VectorAssembler: Combines multiple feature columns (numeric or indexed) into a single vector column called features.
- StandardScaler: Scales features so they have zero mean and/or unit variance.
- OneHotEncoder (convert categorical indices to one-hot vectors)

Estimator

Learns from data and produces a Model.

Examples:

- LogisticRegression
- DecisionTreeClassifier

Pipeline Concept

Raw Data → Feature Engineering → Model → Prediction

- Pipeline ensures reproducibility.

Working Setup Steps for PySpark on Windows

1. Install Compatible Software Versions

Install the following:

Component	Version
Apache Spark	4.1.1
Java	JDK 17
Python	3.11

Even though Spark 4.x is new, the ecosystem around it sometimes lags behind the newest Python versions.

So most production Spark setups still use: Python 3.10 / 3.11

2. Verify Python Installation

Open Command Prompt and verify Python:

```
python --version
```

Expected output:

```
Python 3.11.x
```

Check the Python path:

```
where python
```

Expected result should include:

```
C:\Users\amanp\AppData\Local\Programs\Python\Python311\python.exe
```

3. Navigate to Spark Directory

Open Command Prompt and go to the Spark installation folder:

```
cd C:\spark-4.1.1-bin-hadoop3\bin
```

4 Run pyspark

Run PySpark with the following command:

```
pyspark
```

Test it using below pyspark statement

```
>>> spark.createDataFrame([(1,"a"),(2,"b")],["id","val"]).show()
```

if you see error like below, then go to step 4b:

```
26/03/12 11:03:52 ERROR Executor: Exception in task 0.0 in stage 0.0 (TID 0)
java.io.IOException: Cannot run program "python3": CreateProcess error=2,
The system cannot find the file specified
```

4 b. Start PySpark with Explicit Python Configuration

Run PySpark with the following command:

```
pyspark --conf spark.pyspark.python=python
```

This ensures Spark uses the correct Python interpreter instead of attempting to launch python3.

6. Test Spark Execution

Run a simple PySpark command:

```
spark.createDataFrame([(1,"a"),(2,"b")],["id","val"]).show()
```

Expected output:

```
+---+---+
| id|val|
+---+---+
|  1|  a|
|  2|  b|
+---+---+
```

Final Working Environment

Component	Version
Spark	4.1.1
Python	3.11
Java	17

Using SparkSession in PySpark

```
from pyspark.sql import SparkSession
```

- the standard way to import SparkSession.
- the entry point for working with DataFrames, Datasets, and executing SQL queries in PySpark.
- It encapsulates the SparkContext and provides a unified interface for Spark SQL operations.

SparkSession

- You cannot actually run Spark without a SparkSession.
- The shell simply creates one automatically for you.

This happens when you start PySpark

When you run: `pyspark`
Apache Spark automatically creates:

```
spark   # SparkSession   ,  sc    # SparkContext
```

That is why you can directly run: `spark.range(5).show()`
without writing: `SparkSession.builder.getOrCreate()`
Because Spark already created the session in the background.

SparkSession actually is

In modern Apache Spark (2.x and later), SparkSession is the entry point to Spark.

It provides access to:

- Spark SQL
- DataFrame API
- Dataset API
- SparkContext
- configuration settings
- catalog and table management

SparkSession Think of it like this:

Component	Purpose
SparkSession	Main entry point
SparkContext	Low-level cluster connection
DataFrame	Structured distributed data

When you want to run a PySpark program from a script (test.py), you normally execute it using spark-submit, which is the standard launcher for Apache Spark applications.

```
cd C:\spark-4.1.1-bin-hadoop3\bin
spark-submit test.py                                        [x]
spark-submit --conf spark.pyspark.python=python test.py   [Python fix]
REM :This ensures Spark workers use the correct Python interpreter. (comment)
```

Environment	Need to create SparkSession?
PySpark shell	❌ No
Python script	✅ Yes
Jupyter Notebook	✅ Yes
Spark application	✅ Yes

SparkSession.builder

`SparkSession.builder` constructs the session.

`.appName()` sets the application name.

`.master()` defines the cluster manager (e.g., local[*] for local mode).

`.config()` allows setting Spark configurations.

`.getOrCreate()` returns an existing session or creates a new one.

script1.py

```python
from pyspark.sql import SparkSession
# Create or retrieve an existing SparkSession
spark = (
    SparkSession.builder
    .appName("ExampleApp")
    .master("local[*]")
    .config("spark.some.config.option", "some-value")
    .getOrCreate()
)
# Create a simple DataFrame
data = [("Alice", 25), ("Bob", 30)]
df = spark.createDataFrame(data, ["Name", "Age"])
# Run SQL queries
df.createOrReplaceTempView("people")
result = spark.sql("SELECT Name FROM people WHERE Age > 26")
result.show()
# Stop the session
spark.stop()
```

script1.py : save under /bin , then execute

```
C:\spark-4.1.1-bin-hadoop3\bin>
spark-submit --conf spark.pyspark.python=python script1.py
```

CASE STUDY

Titanic Survival Prediction

- Source: Kaggle Titanic Dataset (titanic.csv)
- Problem Type:Binary Classification
- Goal: Predict passenger survival (0 = No, 1 = Yes)

Titanic Dataset Columns

- PassengerId, - Pclass, - Name, - Sex, - Age
- SibSp, - Parch, - Ticket, - Fare,- Cabin
- Embarked, - Survived (Target)

ML Problem Definition

Predict Passenger Survival (Titanic Dataset)

Goal: Predict whether a passenger survived or not using available features.

Dataset features:

- Passenger Class (Pclass)
- Gender (Sex)
- Age
- Number of siblings/spouses (SibSp)
- Number of parents/children (Parch)
- Fare paid
- Target variable: Survived (0 = No, 1 = Yes)

Step 0: Why Use PySpark for ML?

Why we use Apache Spark:

- Handles large datasets
- Distributed processing
- Built-in ML library
- Scalable pipelines
- Spark ML workflow:
- Data → Feature Engineering → Model Training → Prediction

Step 1: Start Spark Session

Spark applications start with SparkSession.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("TitanicML") \
    .getOrCreate()
```

SparkSession acts as the entry point to Spark.

Step 2 — Load the Dataset

Read the CSV file.

```
df = spark.read.csv(
    "titanic.csv",
    header=True,
    inferSchema=True
)
df.show(5)
```

Example output:

```
+-----------+--------+------+--------------------+------+----+-----+-----+----------------+-------+-----+--------+
|PassengerId|Survived|Pclass|                Name|   Sex| Age|SibSp|Parch|          Ticket|   Fare|Cabin|Embarked|
+-----------+--------+------+--------------------+------+----+-----+-----+----------------+-------+-----+--------+
|          1|       0|     3|Braund, Mr. Owen ...|  male|22.0|    1|    0|       A/5 21171|   7.25| NULL|       S|
|          2|       1|     1|Cumings, Mrs. Joh...|female|38.0|    1|    0|        PC 17599|71.2833|  C85|       C|
```

Step 3 — Data Cleaning

Real datasets contain missing values.

Remove incomplete records.

```
df = df.select("Survived","Pclass","Sex","Age","SibSp","Parch","Fare")

df = df.dropna()
```

Purpose:

- Remove null values
- Keep relevant columns

Step 4 — Convert Categorical Data

Machine learning algorithms require numeric data.

Convert text → numeric using StringIndexer.

```
from pyspark.ml.feature import StringIndexer

sex_indexer = StringIndexer(
    inputCol="Sex",
    outputCol="SexIndex"
)
```

Example transformation:

male → 1

female → 0

Step 5 — Create Feature Vector

ML models require all features in a single vector.

Use VectorAssembler.

```
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(
    inputCols=[
        "Pclass","SexIndex",
        "Age","SibSp","Parch","Fare"
    ],
    outputCol="features"
)
```

Result:

features = [3,1,22,1,0,7.25]

Step 6 — Train Linear Regression Model

We train a **Linear Regression model.

```
from pyspark.ml.regression import LinearRegression

lr = LinearRegression(
    featuresCol="features",
    labelCol="Survived"
)
```

Model learns the relationship between:

features → survival probability

Step 7 — Create ML Pipeline

A pipeline automates all steps.

```
from pyspark.ml import Pipeline
pipeline = Pipeline(
    stages=[
        sex_indexer,
        assembler,
        lr
    ]
)
```

Workflow becomes:

Raw Data → Indexer → Feature Vector → Linear Regression Model

Step 8 — Train and Predict

Split dataset into training and testing sets.

```
train, test = df.randomSplit([0.8,0.2], seed=42)
model = pipeline.fit(train)
predictions = model.transform(test)
predictions.select("Survived","prediction").show()
```

Example output:

```
+--------+--------------------+
|Survived|          prediction|
+--------+--------------------+
|       0| 0.8524357063025458|
|       0| 0.5818417854232223|
```

Prediction close to 1 → likely survived.

Step 9 a. Evaluation matrix (R^2, RMSE, and MAE)

```
from pyspark.ml.evaluation import RegressionEvaluator

evaluator_rmse = RegressionEvaluator(labelCol="Survived", predictionCol="prediction", metricName="rmse")
evaluator_mae = RegressionEvaluator(labelCol="Survived", predictionCol="prediction", metricName="mae")
evaluator_r2  = RegressionEvaluator(labelCol="Survived", predictionCol="prediction", metricName="r2")

rmse = evaluator_rmse.evaluate(predictions)
mae  = evaluator_mae.evaluate(predictions)
r2   = evaluator_r2.evaluate(predictions)

print(f"RMSE: {rmse}")
print(f"MAE: {mae}")
print(f"R²: {r2}")
```

9 b. Confusion Matrix

```
from pyspark.sql.functions import col

# Cast both columns to integer for consistency
cm_df = predictions.select(col("Survived").cast("int"), col("prediction").cast("int"))

# Compute counts
tp = cm_df.filter((col("Survived") == 1) & (col("prediction") == 1)).count()
tn = cm_df.filter((col("Survived") == 0) & (col("prediction") == 0)).count()
fp = cm_df.filter((col("Survived") == 0) & (col("prediction") == 1)).count()
fn = cm_df.filter((col("Survived") == 1) & (col("prediction") == 0)).count()

print("Confusion Matrix:")
print("              Predicted 0    Predicted 1")
print(f"Actual 0      {tn:12}    {fp:12}")
print(f"Actual 1      {fn:12}    {tp:12}")
```

Step 9 c. Evaluation matrix (Correlation Heatmap)

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Apply the StringIndexer so SexIndex exists
df_indexed = sex_indexer.fit(df).transform(df)

# Now select from df_indexed, not df
pdf = df_indexed.select("Survived","Pclass","SexIndex","Age","SibSp","Parch","Fare").toPandas()

# Compute correlation matrix
corr = pdf.corr()

# Plot heatmap
plt.figure(figsize=(8,6))
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap - Titanic Features")
plt.show()
```

📉 MAE (Mean Absolute Error): Average of the absolute differences between predicted and actual values.

Interpretation: Easier to understand than RMSE because it’s in the same units as the target variable. Lower MAE = better fit.

In Titanic context: tells you the average “off by” amount in survival prediction.

📉 RMSE (Root Mean Squared Error): Square root of the average squared differences between predicted and actual values.

Interpretation: Penalizes larger errors more heavily. Lower RMSE = better fit.

In Titanic context: Shows how far, on average, your predictions deviate from the actual survival labels.

📈 R^2 (Coefficient of Determination): Measures how well the model explains the variance in the target variable.

Range: 0 to 1 (sometimes negative if the model is worse than a baseline).

Interpretation:

$R^2 = 1 \rightarrow$ Perfect fit.

$R^2 = 0 \rightarrow$ Model explains none of the variance.

In Titanic context: Since survival is binary, R^2 is less intuitive, but it still shows how much of the variation in survival outcomes is captured by your linear regression model.

📊 Confusion Matrix: A table that compares actual labels with predicted labels in classification problems.

Structure:

- True Positive (TP): Model predicts “Survived” and passenger actually survived.
- True Negative (TN): Model predicts “Did Not Survive” and passenger actually did not survive.
- False Positive (FP): Model predicts “Survived” but passenger did not survive.
- False Negative (FN): Model predicts “Did Not Survive” but passenger survived.

Use: Helps you see not just accuracy, but the types of errors the model makes (e.g., missing survivors vs. wrongly predicting survival).

🔥 Correlation Heatmap: A visual representation of the correlation coefficients between features.

Correlation coefficient (r):

Range: -1 to +1

+1 → Perfect positive correlation (both increase together).

-1 → Perfect negative correlation (one increases, the other decreases).

0 → No linear relationship.

Use: Helps identify relationships between features (e.g., Sex vs. Survival, Fare vs. Survival).

In Titanic context: You'll likely see strong correlation between SexIndex and Survived (women had higher survival rates), and some correlation with Pclass and Fare.

Stop Spark

```
spark.stop()
```

Workflow Summary

```
Titanic Dataset
       ↓
Data Cleaning
       ↓
Categorical Encoding
       ↓
Feature Vector Creation
       ↓
Linear Regression Model
       ↓
Prediction
```

Running Python Spark

```
C:\Users\amanp> pyspark --conf spark.pyspark.python=python

>>>
```

pyspark → Python shell for Spark (PySpark).

Step 1: Start Spark Session

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("TitanicML") \
    .getOrCreate()
```

- Modern, unified entry point for Apache Spark (introduced in 2.0)
- Acts as a wrapper around the lower-level SparkContext, SQLContext, and HiveContext
- While you might be able to execute commands directly.
- Variable is pre-initialized), the SparkSession object is essential for structuring, configuring, and managing your application's interaction with the cluster.

Step 2: Load Titanic Dataset

```
df = spark.read.csv("titanic.csv", header=True, inferSchema=True)
df.printSchema()
df.show(5)
```

Step 3: Data Cleaning

Handle missing values:

```
df = df.dropna(subset=["Age", "Embarked"])
```

Embarked= locations: C for Cherbourg, Q for Queenstown, and S for Southampton

Step 4: Encode Categorical Features

```
from pyspark.ml.feature import StringIndexer

sex_indexer = StringIndexer(inputCol="Sex", outputCol="SexIndex")
embarked_indexer = StringIndexer(inputCol="Embarked", outputCol="EmbarkedIndex")
```

Step 5: Feature Assembly

```
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(
    inputCols=["Pclass","SexIndex","Age","SibSp","Parch","Fare","EmbarkedIndex"],
    outputCol="features"
)
```

Step 6: Build Pipeline

```
from pyspark.ml import Pipeline

pipeline = Pipeline(stages=[sex_indexer, embarked_indexer, assembler])
model_pipeline = pipeline.fit(df)
df = model_pipeline.transform(df)
```

Step 7: Train-Test Split

```
train, test = df.randomSplit([0.8, 0.2], seed=42)
```

Logistic Regression

```
from pyspark.ml.classification import LogisticRegression

lr = LogisticRegression(featuresCol="features", labelCol="Survived")
model = lr.fit(train)
predictions = model.transform(test)
```

- The warnings are about output display and performance optimizations, not errors.
- You can continue working with your predictions DataFrame.

Decision Tree Classifier

```
from pyspark.ml.classification import DecisionTreeClassifier

dt = DecisionTreeClassifier(featuresCol="features", labelCol="Survived")
dt_model = dt.fit(train)
dt_predictions = dt_model.transform(test)
```

Support Vector Machine

```
from pyspark.ml.classification import LinearSVC

svm = LinearSVC(featuresCol="features", labelCol="Survived")
svm_model = svm.fit(train)
```

Naive Bayes

```
from pyspark.ml.classification import NaiveBayes

nb = NaiveBayes(featuresCol="features", labelCol="Survived")
nb_model = nb.fit(train)
```

Confusion Matrix

```
predictions.groupBy("Survived", "prediction").count().show()
```

Classification Evaluation Metrics

- Accuracy
- Precision
- Recall
- F1 Score
- AUC

```
from pyspark.ml.evaluation import BinaryClassificationEvaluator

evaluator = BinaryClassificationEvaluator(labelCol="Survived")
auc = evaluator.evaluate(predictions)
```

Regression Example (Fare Prediction)

Predict passenger Fare using other features.

```
from pyspark.ml.regression import LinearRegression

lr_reg = LinearRegression(featuresCol="features", labelCol="Fare")
reg_model = lr_reg.fit(train)
reg_predictions = reg_model.transform(test)
```

Regression Metrics

```
from pyspark.ml.evaluation import RegressionEvaluator

evaluator = RegressionEvaluator(labelCol="Fare", metricName="rmse")
rmse = evaluator.evaluate(reg_predictions)
```

Metrics:

- R2
- RMSE
- MAE

K-Means Clustering

Cluster passengers based on socioeconomic attributes.

```
from pyspark.ml.clustering import KMeans

kmeans = KMeans(k=3, featuresCol="features")
cluster_model = kmeans.fit(df)
clustered = cluster_model.transform(df)
```

Correlation Heatmap

```
import seaborn as sns
import matplotlib.pyplot as plt

pandas_df = df.select("Pclass","Age","Fare","SibSp","Parch","Survived").toPandas()
sns.heatmap(pandas_df.corr(), annot=True)
plt.show()
```

Model Comparison Strategy

1. Train multiple models
2. Compare Accuracy & AUC
3. Analyze confusion matrix
4. Select best model

Saving Best Model

```
model.write().overwrite().save("titanic_best_model")
```

Loading Model

```
from pyspark.ml.classification import LogisticRegressionModel

loaded_model = LogisticRegressionModel.load("titanic_best_model")
```

Step-by-Step Lab Practical

Lab 1:

- Load Titanic dataset
- Clean missing values
- Encode categorical variables
- Train Logistic Regression
- Evaluate performance

Lab 2:

- Train Decision Tree
- Compare with Logistic Regression

Lab 3:

Assignment Questions

1. Compare LR, DT, SVM performance
2. Apply feature scaling
3. Tune hyperparameters
4. Improve accuracy beyond 80%

Viva Questions

1. What is VectorAssembler?
2. Difference between Estimator and Transformer?
3. What is R2?
4. Explain Confusion Matrix
5. Why use Pipeline in Spark ML?

Final Summary

In this unit we implemented:

- Logistic Regression
- Decision Tree
- SVM
- Naive Bayes
- Linear Regression
- K-Means
- Confusion Matrix
- R2, RMSE, MAE
- Correlation Heatmap
- Full ML Pipeline using Titanic dataset

End of Unit 6

Professor-Level Spark ML using Titanic Dataset